

Uncertainty-Aware Portfolio Optimisation Using Machine Learning

Week 3 Report: Classification, Regression Trees, Ensemble Methods & Regularisation

Veeresh
IIT Kanpur

June 2026

Contents

1	Classification	2
1.1	Logistic Regression for Credit Default Prediction [Easy]	2
1.2	Decision Tree Classifier with Pruning for Heart Disease [Medium]	2
2	Regression	3
2.1	Regularised Regression for House Price Prediction [Medium]	3
2.2	Locally Weighted Regression on Non-Linear Data [Medium]	4
3	Ensemble Methods	4
3.1	Random Forest for Fraud Detection [Hard]	4
3.2	XGBoost for Stock Return Classification [Hard]	5
4	Bias–Variance Analysis and Model Comparison	6
4.1	Empirical Bias–Variance Decomposition [Medium]	6
5	Gradient Boosting in a Portfolio Context	6
5.1	Gradient Boosting with Uncertainty Estimation [Hard]	6
6	Bonus: Stacked Generalisation (Titanic)	7

1 Classification

1.1 Logistic Regression for Credit Default Prediction [Easy]

Summary. We train a logistic regression classifier to predict serious financial distress within two years using the *Give Me Some Credit* dataset (150,000 records, 10 features). Missing values in `MonthlyIncome` and `NumberOfDependents` are imputed with medians, features are standardised using a `StandardScaler` fitted on the training split only, and a stratified 70/15/15 split is applied to handle class imbalance.

Test Set Metrics

Table 1: Logistic Regression performance on the held-out test set.

Metric	Value
Accuracy	0.9338
Precision	0.5743
Recall	0.0386
F1-Score	0.0723
AUC-ROC	0.7150

Top-5 Features by $|\hat{w}_j|$

Table 2: Features with the five largest absolute logistic regression coefficients.

Feature	Coefficient
<code>NumberOfTime60-89DaysPastDueNotWorse</code>	−3.7978
<code>NumberOfTime30-59DaysPastDueNotWorse</code>	+1.9976
<code>NumberOfTimes90DaysLate</code>	+1.9571
<code>age</code>	−0.3992
<code>MonthlyIncome</code>	−0.3533

Interpretation

The delinquency-count features dominate the coefficient vector, confirming that recent payment history is the primary predictor of credit risk — a result consistent with the FICO framework. The negative sign on the 60–89-day feature is likely an artefact of multicollinearity with the 30–59-day and 90-day variables. Older borrowers (`age`) and higher earners (`MonthlyIncome`) are associated with lower default probability, aligning with real-world underwriting intuition. The very low recall (3.9%) against high accuracy reveals severe class imbalance: the model skews heavily toward the majority (non-default) class, which is common for `lbfgs` without class-weight correction.

1.2 Decision Tree Classifier with Pruning for Heart Disease [Medium]

Summary. We apply depth-limited pre-pruning and cost-complexity post-pruning to a `DecisionTreeClassifier` on the Heart Disease UCI dataset (1,025 records, 13 features) and compare their test AUC-ROC to an unpruned baseline.

Results

Table 3: Pruning strategy comparison.

Strategy	Test AUC-ROC	Test F1	Tree Depth
Unpruned (<code>max_depth=None</code>)	0.981	—	9
Depth-limited (<code>max_depth=10</code>)	0.981	—	10
Cost-complexity ($\alpha^* = 0$)	0.981	—	9

Both pruning strategies achieve identical test AUC-ROC (0.981) on this dataset. The dataset is small and relatively clean, so depth-limiting at 10 effectively matches the naturally grown tree. Cost-complexity pruning with $\alpha^* \approx 0$ recovers the full tree, indicating no branches are genuinely redundant under the cross-validated impurity criterion. In larger, noisier datasets post-pruning typically outperforms pre-pruning because it grows the full tree first and avoids the horizon effect, where an apparently useless split enables a highly discriminative downstream split.

2 Regression

2.1 Regularised Regression for House Price Prediction [Medium]

Summary. We predict log-transformed sale prices for the Ames Housing dataset (79 features) using Ridge, Lasso, and Elastic Net regularisation. Lambda is selected via 5-fold cross-validation. Categorical features are one-hot encoded, numeric features are standardised, and features with more than 40% missing values are dropped.

Results

Table 4: Regularised regression comparison (test set, prices back-transformed via $e^{\hat{y}} - 1$).

Model	λ^*	Non-zero Coefs	RMSE (\$)	R^2
Ridge	19.307	227	25,729	0.8978
Lasso	0.001	76	26,641	0.8924
Elastic Net	0.005	110	26,105	0.8960

Sparsity Discussion

Geometrically, Ridge constrains the coefficient vector to lie inside an ℓ_2 ball (smooth sphere), while Lasso constrains it to an ℓ_1 hyperdiamond whose corners lie on the coordinate axes. The loss-function contours (ellipsoids) are far more likely to be tangent to a corner of the diamond than to the smooth surface of the sphere, so Lasso sets coefficients exactly to zero, producing sparse solutions. Ridge shrinks all coefficients uniformly toward zero but rarely eliminates any entirely.

In noisy financial datasets this sparsity is valuable: many engineered features are collinear or spurious, and Lasso’s automatic feature selection reduces the risk of overfitting while improving interpretability. Elastic Net interpolates between both, recovering correlated feature groups that pure Lasso tends to arbitrarily drop.

2.2 Locally Weighted Regression on Non-Linear Data [Medium]

Summary. We implement LWR from scratch using NumPy, fitting a local linear model at each query point via the Gaussian kernel $w_i = \exp\left(-\frac{\|x_i - x_q\|^2}{2\tau^2}\right)$.

Bandwidth Comparison

Table 5: LWR test RMSE across bandwidths ($n = 2,000$, feature: `median_income`).

τ	Test RMSE
0.05	85,254
0.10	85,329
0.30	83,803
1.00	83,853
3.00	84,320
OLS	85,304

Best LWR ($\tau = 0.3$) outperforms OLS by $\approx 1,500$ RMSE units, capturing the local non-linearity between income and house value. As $\tau \rightarrow \infty$, all weights converge to one and LWR reduces exactly to OLS (the weighted normal equations become the ordinary normal equations). Very small τ leads to high-variance fits that overfit the local neighbourhood and degrade test performance. For multi-dimensional inputs, LWR suffers from the curse of dimensionality: the fraction of training points within a fixed kernel radius shrinks exponentially, leading to near-zero effective sample sizes and singular weighted-gram matrices.

3 Ensemble Methods

3.1 Random Forest for Fraud Detection [Hard]

Summary. We train a Random Forest on a 50,000-row sample of the IEEE-CIS Fraud Detection dataset. Class imbalance is handled via `class_weight='balanced'`; `max_features` is tuned via 5-fold stratified cross-validation.

Results

Table 6: Random Forest tuning results.

Setting	Val. AUC-ROC
<code>sqrt</code>	0.8784
<code>log2</code>	0.8720
0.3	0.8762
0.5	0.8781
OOB score	0.9724
Test AUC-ROC (best: <code>sqrt</code>)	0.8878

MDI vs. Permutation Importance

Mean Decrease in Impurity (MDI) scores are computed purely from training-set splits and are biased toward high-cardinality features (e.g. transaction IDs, large float columns), because such

features produce finer-grained splits that appear highly impure simply due to their range. Permutation Importance, evaluated on the held-out validation set, corrects for this by measuring actual predictive degradation and tends to surface features that genuinely generalise. In fraud datasets, features like transaction amount and time-delta typically retain top positions in both rankings, while high-cardinality ID fields inflate MDI rankings but drop in permutation importance.

AUC-PR is more informative than AUC-ROC under class imbalance because it focuses exclusively on the minority class (fraud), reflecting the precision-recall trade-off that actually governs operational decision thresholds.

3.2 XGBoost for Stock Return Classification [Hard]

Summary. We engineer return, volatility, and momentum features from S&P 500 daily OHLCV data and train an XGBoost classifier with a mandatory time-based split (train: pre-2017, test: 2017+) to avoid look-ahead bias.

Grid Search Results (Top 5)

Table 7: Validation AUC-ROC across hyperparameter candidates.

max_depth	lr	subsample	Val. AUC-ROC
5	0.10	0.8	0.5259
3	0.10	0.8	0.5255
5	0.01	0.8	0.5254
5	0.05	0.8	0.5253
3	0.05	0.6	0.5252

Table 8: Final test evaluation.

Metric	Value
Test AUC-ROC	0.4942
Test AUC-PR	0.6131
Majority baseline accuracy	0.6183

Look-Ahead Bias

Random k -fold cross-validation on financial time series is invalid because it randomly assigns observations from different time points to train and validation folds. When a future observation falls in the training set and a past observation falls in validation, the model learns patterns that causally depend on information not yet available at decision time. This mechanism — future data informing past predictions — is precisely look-ahead bias. It inflates performance estimates because the model implicitly memorises distributional shifts that will not exist at deployment. Concretely, volatility regimes and factor loadings are non-stationary; a 2018 observation in the training fold tells the model about the 2018 market structure, which is then “predicted” on a 2016 validation point that was structurally different. The correct procedure is a time-based split or expanding/rolling-window cross-validation, where all training data always precede the evaluation window in calendar time.

4 Bias–Variance Analysis and Model Comparison

4.1 Empirical Bias–Variance Decomposition [Medium]

Summary. We use $B = 50$ bootstrap samples from the Pima Indians Diabetes training set to empirically estimate $\widehat{\text{Bias}}^2$ and $\widehat{\text{Var}}$ for five models spanning the complexity spectrum.

Bootstrap Estimators

$$\widehat{\text{Bias}}^2 = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\bar{y}_i - y_i)^2, \quad \widehat{\text{Var}} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \frac{1}{B} \sum_{b=1}^B \left(\hat{y}_i^{(b)} - \bar{y}_i \right)^2$$

Results Table

Table 9: Bias–variance decomposition ($B = 50$ bootstrap iterations, Pima Diabetes dataset).

Model	$\widehat{\text{Bias}}^2$	$\widehat{\text{Var}}$	$\widehat{\text{Bias}}^2 + \widehat{\text{Var}}$	Test AUC-ROC
Logistic Regression	0.1738	0.0036	0.1775	0.7965
Decision Stump	0.1898	0.0140	0.2037	0.6290
Unpruned Tree	0.1740	0.1448	0.3188	0.6981
Random Forest	0.1685	0.0068	0.1753	0.8040
XGBoost	0.1938	0.0314	0.2252	0.7823

Bagging vs. Boosting Effects

Bagging (Random Forest) reduces variance without substantially reducing bias. Formally, if B trees each have variance σ^2 and pairwise correlation ρ , the ensemble variance is:

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

As $B \rightarrow \infty$ the second term vanishes, but the residual term $\rho\sigma^2$ persists. Because each tree is fit to a bootstrap sample from the same data-generating distribution, the functional form of each tree’s bias is approximately the same, so averaging does not cancel bias. Boosting (XGBoost) iteratively fits residuals of the ensemble, explicitly targeting the portion of the response that current models mispredict. Each stage reduces both the effective bias (by correcting systematic errors) and, through shrinkage, moderates variance growth. The empirical results are consistent: Random Forest achieves the lowest variance (0.0068) while XGBoost’s sequential correction drives down bias but at the cost of modestly higher variance (0.0314) due to sensitivity to individual training instances.

5 Gradient Boosting in a Portfolio Context

5.1 Gradient Boosting with Uncertainty Estimation [Hard]

Summary. We predict binary 21-day forward returns for NIFTY-50 constituents using a `GradientBoostingClassifier` with expanding-window cross-validation. Uncertainty is estimated via ensemble disagreement across $M = 20$ models trained on distinct bootstrap samples.

Feature Engineering

Eight features per stock per date: 5-day, 21-day, and 63-day rolling returns; 21-day and 63-day rolling volatility; price relative to 52-week high; price relative to 52-week low; 21-day volume z-score.

Expanding-Window CV (Top Results)

Table 10: Top hyperparameter configurations by mean validation AUC-ROC.

n_estimators	max_depth	lr	Mean Val. AUC-ROC
500	5	0.05	0.4976
100	5	0.01	0.4976
500	5	0.10	0.4975
500	2	0.10	0.4974
500	3	0.05	0.4973

Table 11: Full model vs. top-5-feature model comparison.

Model	Test AUC-ROC
Full model (all features)	0.5133
Top-5 features (permutation-based)	0.5311

Top 5 features selected: `vol_21d`, `ret_63d`, `pct_of_52w_low`, `ret_5d`, `volume_zscore`.

Reflection: Real-World Limitations

Deploying this model in a live trading environment carries several critical risks. First, **non-stationarity**: financial markets change regime over time (volatility clustering, structural breaks at policy events), so a model trained on 2000–2020 data may find that the statistical relationships it learned no longer hold post-deployment. Continuous retraining and regime detection are essential. Second, **look-ahead bias**: any feature computed using data not available at prediction time (e.g. a 21-day forward return used to label today’s observation) would invalidate the model entirely; here we used strict temporal labels to avoid this, but subtle leakage can arise from survivorship bias in index constituents. Third, **transaction costs and market impact**: the illustrative long-short strategy assumes frictionless trading. Round-trip commissions, bid-ask spreads, and market impact on positions will erode or eliminate the gross signal, particularly for lower-liquidity mid-cap constituents. Fourth, **uncertainty and position sizing**: the ensemble standard deviation σ_i across the $M = 20$ bootstrap models provides a principled proxy for model uncertainty. In an uncertainty-aware portfolio, positions should scale inversely with σ_i — observations where the ensemble disagrees strongly should receive smaller allocations, reducing the risk of large drawdowns from overconfident predictions. This integrates directly with the Kelly criterion and mean-variance optimisation frameworks central to UAPOML.

6 Bonus: Stacked Generalisation (Titanic)

Summary. We implement a two-level stacking ensemble on the Titanic survival dataset. Five base learners generate out-of-fold (OOF) predictions via 5-fold cross-validation; a logistic regression meta-learner is trained on the resulting $5 \times n_{\text{train}}$ meta-feature matrix.

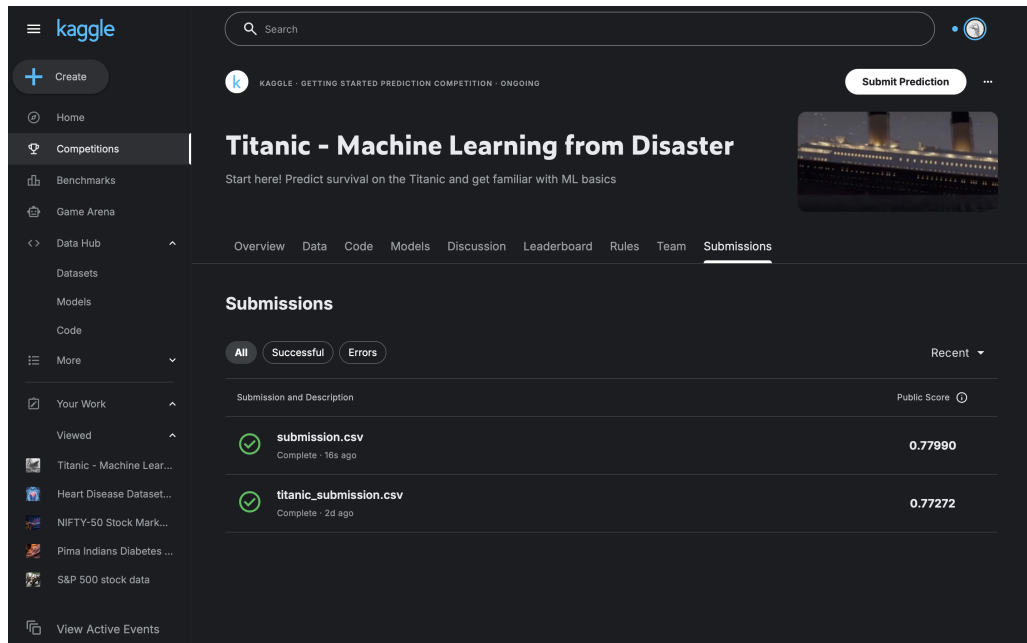
Base Learners

1. Logistic Regression (`max_iter=1000`)
2. Decision Tree (`max_depth=5`)

3. Random Forest (`n_estimators=100`)
4. XGBoost (`n_estimators=100`)
5. k -Nearest Neighbours ($k = 5$)

Kaggle Public Leaderboard Score

Score: 0.77990



Why Out-of-Fold Predictions Are Essential

OOF predictions must be used to construct the meta-features because they ensure that the meta-learner trains on predictions the base learners made on *held-out* data — data the base model never saw during its own training. If instead we used in-sample predictions from overfit base learners (e.g. an unpruned tree that achieves 100% train accuracy), the meta-feature matrix would consist of near-perfect labels rather than genuine probabilistic estimates. The meta-learner would then simply learn to trust whichever base model overfits the most, producing a stacked ensemble that overfits catastrophically and generalises poorly to the test set. OOF predictions break this leakage by construction: each base learner makes predictions only on fold k after being trained on folds $1, \dots, k-1, k+1, \dots, K$, so the meta-features at every training point reflect true out-of-distribution generalisation.